

# X Desenvolvimento Web com .NET

Com desenvolvimento Web com .NET para solucionar o problema da empresa em encontrar funcionários capacitados para os respectivos cargos das suas lojas espalhadas por São Paulo, resolvemos criar um painel administrativo para a postagem de vagas da empresa, visualização dos candidatos e uma API Restful para atender a nossa plataforma mobile.

Para a construção do painel administrativo utilizamos o padrão MVC com a linguagem de programação C# e o framework .NET. O padrão MVC facilita a separação de responsabilidades dentro do código, dividindo as regras de negócio, a parte visual e a apresentação dos dados em camadas diferentes no programa. Ele divide a aplicação em três partes principais:

- Model: Representando todas as regras de negócios e a manipulação dos dados, interagindo com o banco de dados e fornecendo os dados para a View.
- View: Responsável pela apresentação visual dos dados ao usuário, sendo interativa e dinâmica.
- Controller: Intermediário entre o Model e a View, processando as entradas do usuário, interagindo com o Model e retornando para a View.

Para o desenvolvimento da API Restful, que é uma interface que dois sistemas de computador utilizam para trocar informações, podendo pegar dados (GET), postar dados (POST), atualizar (PUT) e deletar (DELETE), para a sua construção utilizamos a Minimal API do ambiente .NET, pois simplificam a criação delas e a forma que configuramos.

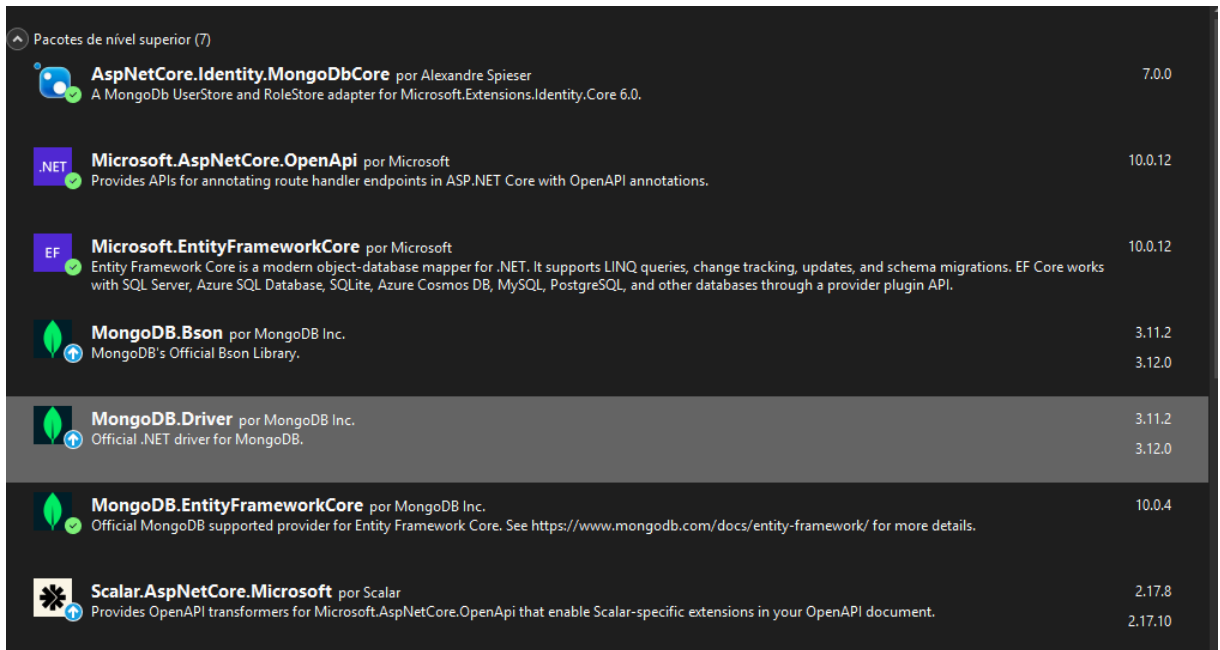
## X.1 Criação do painel administrativo em padrão MVC

O painel administrativo foi criado para realizar as seguintes funcionalidades: realizar o Log In do recrutador, adicionar recrutadores, detalhes das vagas, visualizar todas as vagas, seus candidatos e os detalhes desses mesmos candidatos.

Com o objetivo de não deixar este documento muito longo, iremos abstrair partes do código, caso tenha interesse em vê-lo completo, está disponível em [https://github.com/vitoriafreitasdev/C\\_sharp\\_estudo/tree/main/mvc/contratacoesWeb](https://github.com/vitoriafreitasdev/C_sharp_estudo/tree/main/mvc/contratacoesWeb).

As bibliotecas que adotamos para o desenvolvimento do sistema, parte MVC e API, foram as seguintes:

Figura 1 - Bibliotecas utilizadas no desenvolvimento



| Pacotes de nível superior (7)                                                      |                                                                                                                                                                                                                                                                                                                                                       |                   |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
|   | <b>AspNetCore.Identity.MongoDbCore</b> por Alexandre Spieser<br>A MongoDB UserStore and RoleStore adapter for Microsoft.Extensions.Identity.Core 6.0.                                                                                                                                                                                                 | 7.0.0             |
|   | <b>Microsoft.AspNetCore.OpenApi</b> por Microsoft<br>Provides APIs for annotating route handler endpoints in ASP.NET Core with OpenAPI annotations.                                                                                                                                                                                                   | 10.0.12           |
|   | <b>Microsoft.EntityFrameworkCore</b> por Microsoft<br>Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server, Azure SQL Database, SQLite, Azure Cosmos DB, MySQL, PostgreSQL, and other databases through a provider plugin API. | 10.0.12           |
|   | <b>MongoDB.Bson</b> por MongoDB Inc.<br>MongoDB's Official Bson Library.                                                                                                                                                                                                                                                                              | 3.11.2<br>3.12.0  |
|   | <b>MongoDB.Driver</b> por MongoDB Inc.<br>Official .NET driver for MongoDB.                                                                                                                                                                                                                                                                           | 3.11.2<br>3.12.0  |
|   | <b>MongoDB.EntityFrameworkCore</b> por MongoDB Inc.<br>Official MongoDB supported provider for Entity Framework Core. See <a href="https://www.mongodb.com/docs/entity-framework/">https://www.mongodb.com/docs/entity-framework/</a> for more details.                                                                                               | 10.0.4            |
|  | <b>Scalar.AspNetCore.Microsoft</b> por Scalar<br>Provides OpenAPI transformers for Microsoft.AspNetCore.OpenApi that enable Scalar-specific extensions in your OpenAPI document.                                                                                                                                                                      | 2.17.8<br>2.17.10 |

A AspNetCore.Identity.MongoDbCore foi usada para a funcionalidade de autenticar e autorizar um usuário. Segue a parte da configuração no [Program.cs](#):

```
builder.Services.AddIdentity<AplicacaoUser, Roles>()
    .AddMongoDbStores<AplicacaoUser, Roles, Guid>(
        mongoDbSettings.ConnectionString,
        mongoDbSettings.DatabaseName
    );
```

// Algumas linhas abaixo do código mostrado acima

```
app.UseAuthentication();
app.UseAuthorization();
```

Como exemplo de autorização mostraremos esta parte do Controller em que apenas usuários com a Roles (Que seriam os papéis daquele usuário dentro do sistema.) de recrutador poderiam realizar a função de visualizar o formulário para adicionar outro recrutador.

```
[Authorize(Roles = "Recrutador")]
[HttpGet]
public async Task<ActionResult> AdicionarRecrutador()
{
    return View();
}
```

```
}
```

Para um exemplo de autenticação temos essa função que está na pasta de Services no arquivo PainelAdminService.cs que verifica se o usuário está logado ou não:

```
public bool UsuarioEstaLogado()
{
    var context = httpContextAccessor.HttpContext;
    var logado = context?.User.Identity?.IsAuthenticated;
    if (logado == true) return true;
    return false;
}
```

A Microsoft.EntityFrameworkCore e MongoDB.EntityFrameworkCore foram utilizadas para conexão com o banco de dados. Escolhemos o MongoDB por ser um banco não relacional e para a criação desse sistema, ele facilitou muito a criação dos atributos necessários para os Models sem ter que realizar a criação de diversas tabelas e a relação entre elas, como seria feito, caso o programa fosse desenvolvido com um banco relacional.

A conexão ao banco foi realizada na seguinte parte do código dentro da pasta Data no arquivo AplicacaoDbContext.cs:

```
using contratacoesWeb.Models;
using Microsoft.EntityFrameworkCore;
using MongoDB.EntityFrameworkCore.Extensions;
namespace contratacoesWeb.Data;
public class AplicacaoDbContext : DbContext
{
    public DbSet<Recrutadores> Recrutadores { get; init; }
    public DbSet<Candidatos> Candidatos { get; init; }
    public DbSet<Vagas> Vagas { get; init; }
    public AplicacaoDbContext(DbContextOptions options) : base(options) { }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<Recrutadores>().ToCollection("Recrutadores");
        modelBuilder.Entity<Candidatos>().ToCollection("Candidatos");
        modelBuilder.Entity<Vagas>().ToCollection("Vagas");
    }
}
```

```
}  
}
```

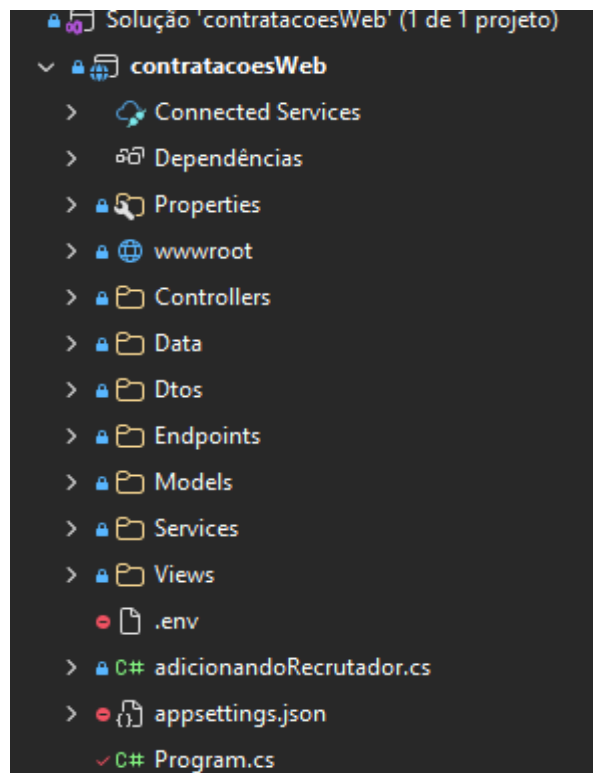
Sendo adicionado no [Program.cs](#) desta maneira:

```
builder.Services.AddDbContext<AplicacaoDbContext>(options =>  
{  
    options.UseMongoDB(mongoDbSettings.ConnectionString,  
mongoDbSettings.DatabaseName);  
});
```

O MongoDB.Bson e MongoDB.Driver foi utilizado no desenvolvimento inicial, mas foi substituído pelo EntityFrameworkCore na versão atual. E os pacotes OpenApi e Scalar foram usados na criação da API.

O programa ficou separado em pastas da seguinte forma:

Figura 2 - Organização de pastas do projeto



Na pasta Controller se encontra o nosso PainelAdminController.cs, nele ficam as funcionalidades do painel administrador, é ele quem chama o PainelAdminService.cs para realizar boa parte lógica dos códigos e acessar ao banco de dados, retornando o resultado para View, segue o exemplo:

```
[Authorize(Roles = "Recrutador")]
```

```
[HttpGet]
```

```
public async Task<IActionResult> AbaAdmin()
{
    var estaLogado = _painelAdminService.UsuarioEstaLogado();
    if (estaLogado == false) return RedirectToAction("Index");
    List<Vagas> vagas = await _painelAdminService.VisualizarVagas();
    return View(vagas);
}
```

Neste código mostrado acima é o retorno da página principal do painel, primeiro verifica se o usuário está logado, caso não esteja retorna para página de login, caso esteja logado irá retornar todas as vagas dentro do banco para a View.

Deixamos da pasta Data a conexão com banco de dados, como já foi mostrado anteriormente. Dentro da Dtos está os nossos objeto de transferência de dados, Endpoints é onde se localiza a nossa Minimal Api, Services está o nosso PainelAdminService.cs, Models possui todos os Models da aplicação, por fim, a pasta Views armazena todas as Views do sistema, desde da página de login até os detalhes dos candidatos. Na próxima sessão iremos mostrar como essas Views se parecem e as funcionalidades presentes nelas.

## X.2 Imagens da tela do painel administrativo

Na tela inicial do painel administrativo se localiza a View de login dos recrutadores para então, após o login, possam ter acesso às funcionalidades do painel, abaixo você pode verificar o resultado final:

Figura 3 - View de login

Para fazer o login preencha o formulário abaixo e clique em "Log In"

Email

Senha

Log In

Na tela principal da área administrativa, os usuários autorizados e identificados podem adicionar vaga, adicionar um recrutador, ver os detalhes das vagas e os seus candidatos:

Figura 4 - View da página principal do painel

Adicionar vaga

Adicionar recrutador

Visualizar vagas

**Desenvolvedor C**

Modelo: Presencial

Tipo: Pleno

Ver detalhes Candidatos

**Desenvolvedor Web**

Modelo: Híbrido

Tipo: Junior

Ver detalhes Candidatos

**Desenvolvedor Java Junior**

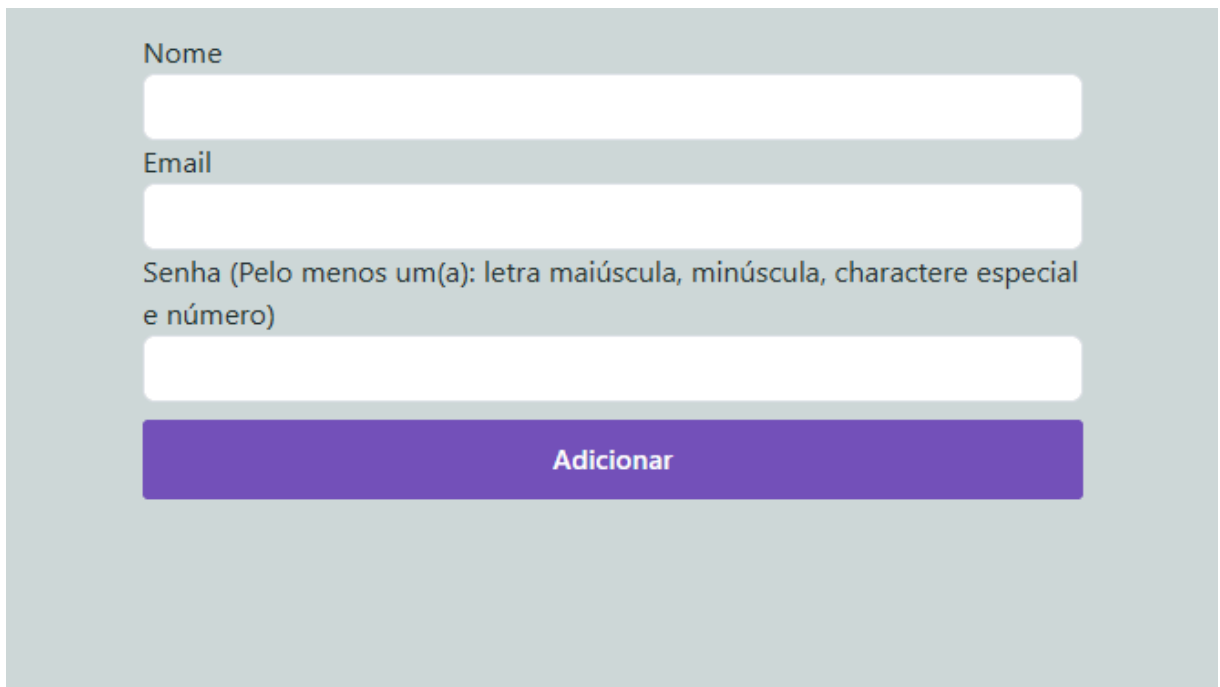
Modelo: Presencial

Tipo: Junior

Ver detalhes Candidatos

O formulário para adicionar o recrutador ficou da seguinte forma:

Figura 5 - View do formulário para adicionar um recrutador



Nome

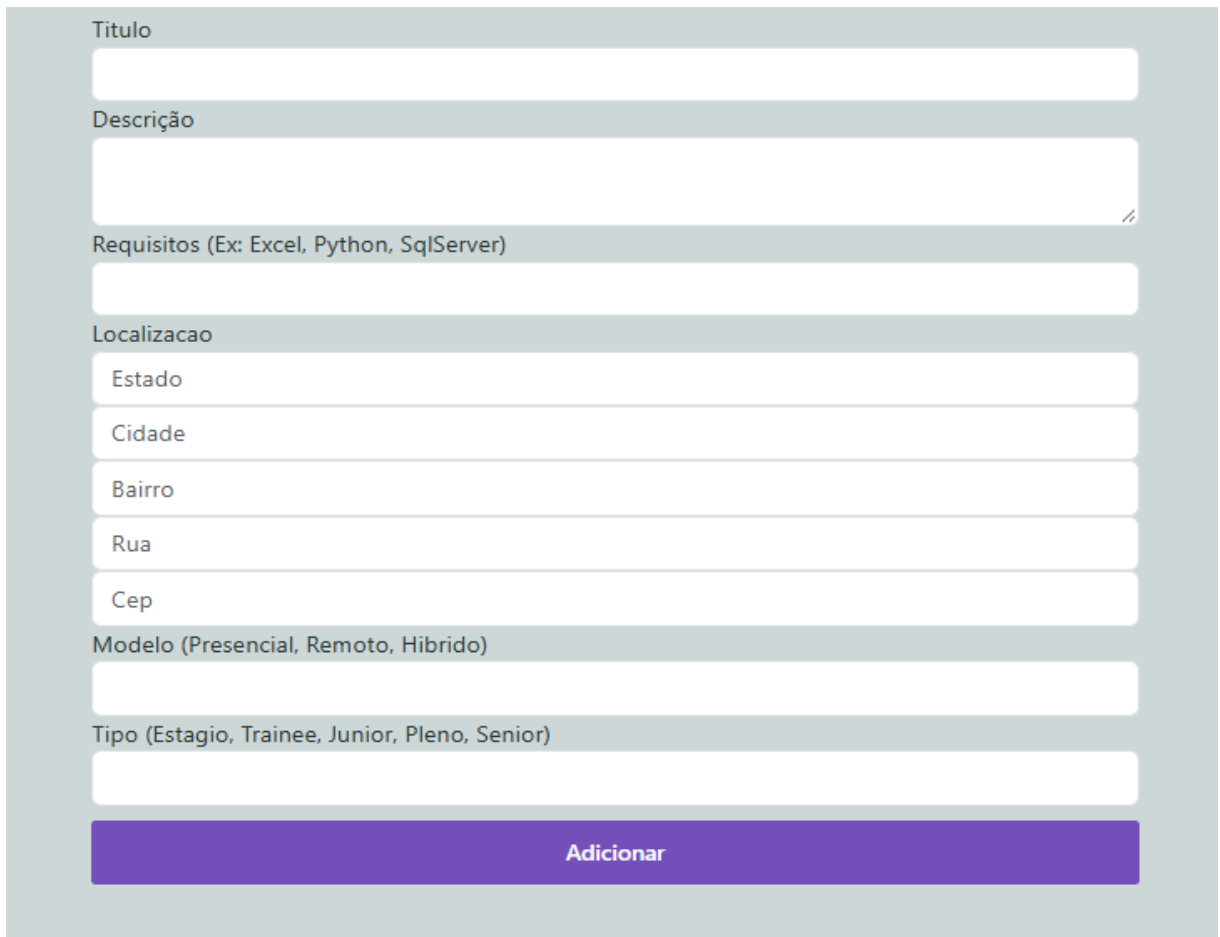
Email

Senha (Pelo menos um(a): letra maiúscula, minúscula, caractere especial e número)

Adicionar

Para o formulário que adiciona uma nova vaga, deixamos ele com um design parecido com os outros formulários.

Figura 6 - View do formulário para adicionar uma vaga



Título

Descrição

Requisitos (Ex: Excel, Python, SqlServer)

Localizacao

Estado

Cidade

Bairro

Rua

Cep

Modelo (Presencial, Remoto, Hibrido)

Tipo (Estagio, Trainee, Junior, Pleno, Senior)

Adicionar

Na próxima imagem é a View que retorna ao clicar em “Ver detalhes” dentro da página principal do painel:

Figura 7 - View de detalhes da vaga

**Desenvolvedor C**

**Modelo:** Presencial **Tipo:** Pleno

---

**Requisitos da vaga**

- ✓ C
- ✓ Hardware
- ✓ Leitura de Datasheets
- ✓ UART
- ✓ I2C
- ✓ SPI
- ✓ GPIOs
- ✓ conversores ADC/DAC

---

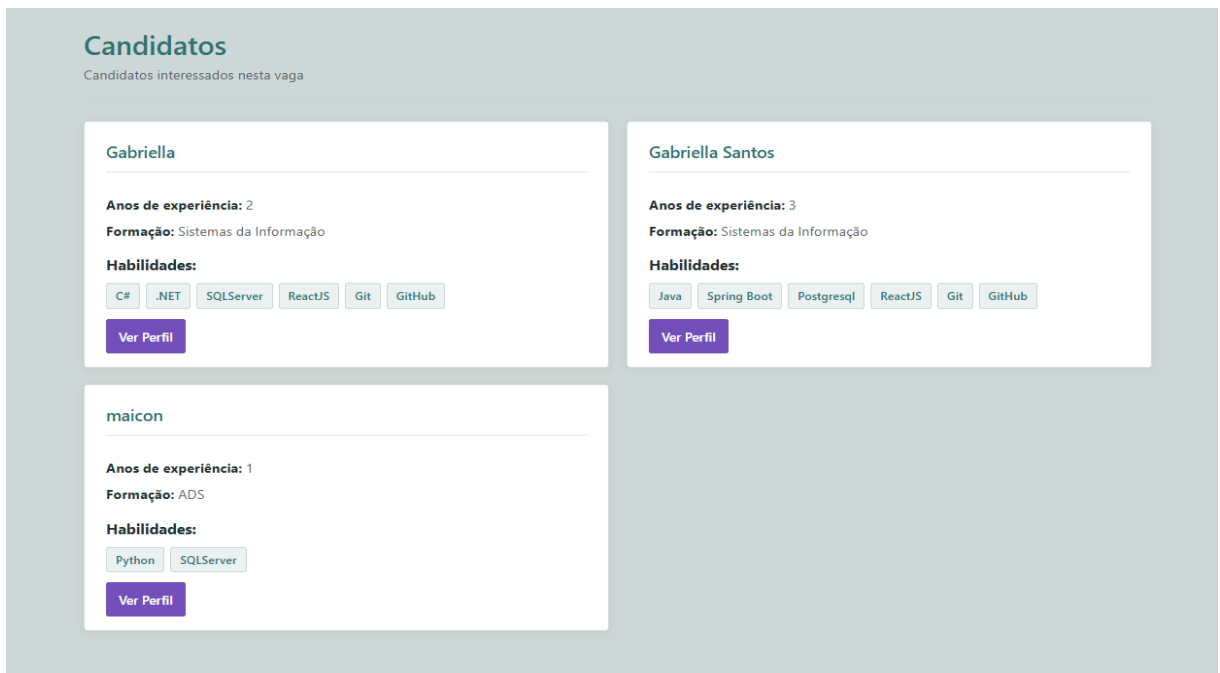
**Localização**

|                            |                           |
|----------------------------|---------------------------|
| <b>Estado</b><br>SÃO PAULO | <b>Cidade</b><br>Barueri  |
| <b>Bairro</b><br>Centro    | <b>Rua</b><br>Santos Neto |
| <b>CEP</b><br>64366-245    |                           |

**Voltar**

Agora na imagem seguinte é a View que retorna ao clicar em “Candidatos” também dentro da página principal do painel:

Figura 8 - View dos candidatos da vaga



Por fim, o resultado ao clicar para “Ver Perfil” da imagem anterior:

Figura 9 - View de detalhes do candidato

**Gabriella**  
Perfil do candidato

**Dados Pessoais**

|                                     |                              |
|-------------------------------------|------------------------------|
| <b>Email</b><br>gabriella@gmail.com | <b>Telefone</b><br>123456789 |
|-------------------------------------|------------------------------|

**Endereço**

|                            |                         |
|----------------------------|-------------------------|
| <b>Estado</b><br>São Paulo | <b>Cidade</b><br>Osasco |
|----------------------------|-------------------------|

**Experiência e Formação**

|                                 |                                                     |
|---------------------------------|-----------------------------------------------------|
| <b>Anos de experiência</b><br>2 | <b>Formação acadêmica</b><br>Sistemas da Informação |
|---------------------------------|-----------------------------------------------------|

**Habilidades**

C# .NET SQLServer ReactJS Git GitHub

**Links profissionais**

LinkedIn

90%

## X.3 Minimal API

A nossa API Restful possui as seguintes funções: retornar todas as vagas disponíveis, retornar dados de uma específica via identificador único, registrar os candidatos, logar os usuários, retornar dados de um candidato, adicionar um candidato em uma vaga e deletar a conta de um.

Os endpoints ficaram desta forma:

Figura 10 - Endpoints da API

| contratacoesWeb                         |      |
|-----------------------------------------|------|
| /api/vagas                              | GET  |
| /api/vagas/{id}                         | GET  |
| /api/candidatos/registrar               | POST |
| /api/candidatos/logar                   | POST |
| /api/candidatos/{id}                    | GET  |
| /api/vagas/{id}<br>/adicionar candidato | PUT  |
| /api/deletarCandidato/{id}              | DEL  |

O código fonte das funções da API ficaram dentro da pasta Endpoints/ApiEndpoints.cs, organizado desta maneira:

```
public static void MapApiEndpoints(this IEndpointRouteBuilder app)
{
    var api = app.MapGroup("/api");
    // Retiramos o código dentro da funções, por motivos de tamanho
    api.MapGet("/vagas", async (AplicacaoDbContext bd) =>
    {
        // Código da funcionalidade
    });

    api.MapGet("/vagas/{id}", async (AplicacaoDbContext bd, ObjectId id) =>
    {
        // Código da funcionalidade
    });

    api.MapPost("/candidatos/registrar", async (AplicacaoDbContext db,
        UserManager<AplicacaoUser> userManager, CandidatoCadastro
candidatoDados) =>
    {
        // Código da funcionalidade
    });
}
```

```

});

        api.MapPost("/candidatos/logar", async (AplicacaoDbContext db,
UserManager<AplicacaoUser> userManager,
        SignInManager<AplicacaoUser> signInManager, [FromBody] LoginCandidato
login) =>
        {
            // Código da funcionalidade
        });

api.MapGet("/candidatos/{id}", async (AplicacaoDbContext db, Objectid id) =>
{
    // Código da funcionalidade
});

        api.MapPut("/vagas/{id}/adicionar candidato", async (AplicacaoDbContext db,
HttpContextAccessor httpContextAccessor,
        Objectid id, Objectid candidatoid) =>
        {
            // Código da funcionalidade
        });

        api.MapDelete("deletarCandidato/{id}", async (AplicacaoDbContext db,
UserManager<AplicacaoUser> userManager, Objectid id) =>
        {
            // Código da funcionalidade
        });
    }
}

```

Parte da configuração no program.cs ficou assim, usando cors para permitir o acesso do mobile para a API:

```
app.MapOpenApi();
```

```
app.MapScalarApiReference(options =>
```

```
{  
    options  
        .WithDefaultHttpClient(ScalarTarget.CSharp, ScalarClient.HttpClient);  
});  
  
app.UseCors(MyAllowSpecificOrigins);  
app.MapApiEndpoints();
```

## X Conclusão final

Com o final do projeto web desenvolvido com .NET, criamos uma plataforma funcional, com espaço para melhora. Como por exemplo, adicionando mais funções ao Controller como filtragem de candidatos dentro da vaga, otimizar a chamada ao banco de dados para deixar tanto o painel, tanto a API ainda mais rápidos.

Em suma, como resultado final, nós temos uma plataforma web escalável, com segurança, que está integrada aos serviços mobile via API Restful.

## links:

<https://medium.com/@gabrielequevedo/model-view-controller-mvc-316fbc169a5>

- data: 26/09/2026

<https://aws.amazon.com/pt/what-is/restful-api/>

- data: 26/09/2026